

Algoritmi e Strutture Dati

Esercizio A1 – Complessità – Punti ≥ 8

Data la forma della ricorrenza, è possibile utilizzare il Master Theorem, versione base.

- Per $a = 2$, abbiamo $\alpha = \log_a 2 = \log_2 2 = 1$, $\beta = 1$; siamo nel secondo caso, $T(n) = \Theta(n \log n)$.
- Per $a = 3$, abbiamo $\alpha = \log_a 2 < 1$, $\beta = 2$; siamo nel terzo caso, $T(n) = \Theta(n^2)$.
- In generale, è possibile dimostrare che $\log_a 2 < 1 < a - 1$ (in quanto decrescente per valori crescenti di a) per tutti i valori $a > 2$; siamo nel terzo caso e si ottiene $T(n) = \Theta(n^{a-1})$.

Valutazione Gli errori più comuni sono stati

- Partendo dall'idea che $T(n) = \Omega(n^{a-1})$, provare a dimostrare che $T(n) = O(n^{a-1})$, il che è vero per tutti i valori $a > 2$, ma non è vero per $a = 2$, in quanto $T(n) = \Theta(n \log n)$, quindi $\Omega(n)$, quindi non può essere $O(n^{a-1}) = O(n)$.
- Assumere che $a = 2$ e paragonarla a Mergesort

Esercizio A2 – AlternaSort – Punti ≥ 10

Versione $O(n \log n)$ Una prima idea consiste nell'ordinare il vettore A in ordine crescente, ottenendo una sequenza ordinata che facilita il posizionamento alternato. Una volta ordinato, si posizionano gli elementi più piccoli (inizio del vettore ordinato) nelle posizioni dispari ($A[1], A[3], A[5], \dots$) e si posizionano gli elementi più grandi (fine del vettore ordinato) nelle posizioni pari ($A[2], A[4], A[6], \dots$).

```
alternaSort(int[] A, int n)
    int[] B = new int[1..n]                                % Crea un vettore B di appoggio
    for k = 1 to n do
        B[k] = A[k]
    sort(B, n)                                              % Ordina il vettore B in ordine crescente
    int i = 1                                              % Inizializza l'indice per le posizioni dispari di A
    int j = n                                              % Inizializza l'indice per le posizioni pari di A
    for k = 1 to n do
        if k mod 2 = 1 then
            A[k] = B[i]                                    % Inserisce il prossimo elemento piccolo nella posizione dispari.
            i = i + 1                                       % Aggiorna l'indice dei valori piccoli
        else
            A[k] = B[j]                                    % Inserisce il prossimo elemento grande nella posizione pari.
            j = j - 1                                       % Aggiorna l'indice dei valori grandi
```

Questa procedura garantisce un ordinamento alternato poiché:

- Gli elementi dispari sono sempre inferiori agli elementi immediatamente successivi
- Gli elementi pari sono sempre maggiori degli elementi immediatamente precedenti

L'algoritmo è corretto perché l'ordinamento iniziale del vettore garantisce che gli elementi più piccoli siano disponibili per le posizioni dispari e quelli più grandi per le posizioni pari. Questo soddisfa il vincolo $A[1] < A[2] > A[3] < A[4] > \dots$. L'assunzione che gli elementi di A siano distinti implica che l'algoritmo non deve gestire conflitti dovuti a valori duplicati. Poiché l'ordinamento iniziale è deterministico, l'algoritmo produce sempre un risultato valido.

La complessità temporale dell'algoritmo è dominata dal passo di ordinamento del vettore B , che ha costo $O(n \log n)$. Il posizionamento degli elementi nelle posizioni dispari e pari è lineare, con costo $O(n)$. Pertanto, la complessità complessiva è:

$$O(n \log n) + O(n) = O(n \log n).$$

Versione $O(n)$ La versione lineare opera facendo scambi locali. Si parte dall'idea che all'iterazione i -esima, i valori fino alla posizione i siano corretti e che si voglia sistemare la posizione $i + 1$ -esima. Ci sono due casi:

- **Caso 1: i è dispari:** In questo caso, vogliamo che $A[i] < A[i + 1]$:
 - Se $A[i] < A[i + 1]$, nessuna azione è necessaria.
 - Se $A[i] > A[i + 1]$, sappiamo che $A[i - 1] > A[i]$ e per transitività $A[i - 1] > A[i + 1]$; scambiando gli elementi $A[i]$ e $A[i + 1]$, otteniamo che $A[i - 1] > A[i] < A[i + 1]$, quindi non intacchiamo la condizione $A[i - 1] > A[i]$.

In entrambi i casi, il vettore è ora alternato fino alla posizione $i + 1$.

- **Caso 2: i è pari:** In questo caso, vogliamo che $A[i] > A[i + 1]$:
 - Se $A[i] > A[i + 1]$, nessuna azione è necessaria.
 - Se $A[i] < A[i + 1]$, sappiamo che $A[i - 1] < A[i]$ e per transitività $A[i - 1] < A[i + 1]$; scambiando gli elementi $A[i]$ e $A[i + 1]$, otteniamo che $A[i - 1] < A[i] > A[i + 1]$, quindi non intacchiamo la condizione $A[i - 1] < A[i]$.

In entrambi i casi, il vettore è ora alternato fino alla posizione $i + 1$.

```

alternasort(int[] A, int n)
  for i = 1 to n - 1 do
    if i mod 2 = 1 then
      if A[i] > A[i + 1] then
        A[i], A[i + 1] = A[i + 1], A[i]           % Scambia gli elementi se A[i] > A[i + 1]
      else
        if A[i] < A[i + 1] then
          A[i], A[i + 1] = A[i + 1], A[i]       % Scambia gli elementi se A[i] < A[i + 1]

```

Il costo in questo caso è lineare.

Valutazione

- Sono state presentate anche soluzione $O(n^2)$, come per esempio avere un indice i che scorre il vettore, scambiando $A[i]$ alternativamente con l'elemento minimo e l'elemento massimo dei valori compresi in $A[i \dots n]$. Questo algoritmo è corretto, ma prende 60% per inefficienza.
- La versione $O(n \log n)$ prende 80%.
- Negli approcci divide et impera, l'approccio più interessante è quello che divide ricorsivamente il vettore in $A[i \dots m]$ e $A[m + 1 \dots j]$, chiama ricorsivamente la funzione sulle due parti ottenendo due vettori alternati, e poi sistema il bordo $A[m]$, $A[m + 1]$ scambiandoli se necessario. Questo algoritmo è associato alla ricorrenza $T(n) = 2T(n/2) + O(1)$, che ha costo lineare.

Esercizio A3 – Flood fill – Punti ≥ 12

Il problema *Flood Fill* può essere interpretato come una visita di un grafo, dove ogni cella della matrice M rappresenta un nodo e le connessioni orizzontali e verticali tra le celle rappresentano gli archi. L'obiettivo è visitare tutte le celle con lo stesso colore iniziale della cella di partenza (x, y) e cambiarle con il colore dato, *color*. Per risolvere il problema, utilizziamo un algoritmo di *breadth-first search* (BFS), che garantisce una visita sistematica dell'area da riempire e previene cicli o visite ridondanti. In particolare, sfruttiamo una coda per gestire l'ordine delle celle da visitare e utilizziamo il fatto che ogni cella, una volta ricolorata, non necessita di essere rivisitata. Questo elimina la necessità di una matrice ausiliaria *visited*.

L'algoritmo si svolge come segue:

- Inizializziamo due array, dr e dc , per rappresentare le quattro mosse possibili: verso l'alto, il basso, la sinistra e la destra. Questi array vengono utilizzati per calcolare le nuove coordinate delle celle adiacenti.
- Memorizziamo il colore iniziale della cella di partenza, *oldcolor*, che determina quali celle devono essere ricolorate.
- Inseriamo la cella iniziale (x, y) nella coda Q .

- Entriamo in un ciclo **while**, continuando finché la coda non è vuota:
 - rimuoviamo una cella (r, c) dalla coda;
 - ricoloriamo la cella corrente con il nuovo colore *color*;
 - per ciascuna delle quattro direzioni definite in *dr* e *dc*, calcoliamo le coordinate (nr, nc) della cella adiacente;
 - se la cella (nr, nc) è all'interno dei limiti della matrice e ha il colore *oldcolor*, la inseriamo nella coda *Q*.
- Terminiamo il processo quando tutte le celle raggiungibili con colore iniziale *oldcolor* sono state ricolorate.

L'algoritmo include un controllo iniziale per evitare operazioni inutili: se il colore *color* è uguale al colore iniziale *oldcolor*, non viene eseguito alcun riempimento.

```

int floodfill(int[][] M, int n, int x, int y, int color)
int[] dr = [-1, 0, +1, 0]                                % Mosse possibili sulle righe
int[] dc = [0, -1, 0, +1]                                % Mosse possibili sulle colonne
int oldcolor = M[x][y]
if color ≠ oldcolor then
  QUEUE Q = Queue()
  Q.enqueue(⟨x, y⟩)
  M[x][y] = color
  while not Q.isEmpty() do                                % Riga, colonna della cella visitata correntemente
    int, int r, c = Q.dequeue()
    for i = 1 to 4 do
      nr = r + dr[i]                                     % Nuova riga
      nc = c + dc[i]                                     % Nuova colonna
      if 1 ≤ nr ≤ n and 1 ≤ nc ≤ n and M[nr][nc] = oldcolor then
        M[nr][nc] = color
        Q.enqueue(⟨nr, nc⟩)

```

L'algoritmo è corretto poiché ogni cella che deve essere ricolorata viene visitata una sola volta. Questo è garantito dal fatto che le celle vengono aggiunte alla coda solo se il loro colore è ancora *oldcolor*. Una volta ricolorata, non sarà più inserita in coda. Il controllo iniziale assicura che nessuna operazione venga eseguita se il colore desiderato è già uguale al colore della cella di partenza.

La complessità dell'algoritmo è determinata dalla visita di un grafo con n^2 nodi, corrispondenti alle celle della matrice, e al massimo $4n^2$ archi, corrispondenti alle connessioni tra celle adiacenti. La complessità temporale è quindi $O(n^2)$, poiché ogni nodo viene visitato una sola volta.

Esercizio B1 – Serie aritmetiche – Punti ≥ 8

Non è necessaria una particolare tecnica algoritmica, solo buon senso.

```

countArithmetic(int[] A, int n)
int series = 0
int lastDiff = +∞
for i = 2 to n do
  int currentDiff = A[i] - A[i - 1]
  if currentDiff ≠ lastDiff then
    lastDiff = currentDiff
    count = 1
  else
    count = count + 1
    if count == 2 then
      series = series + 1
return series

```

1. Inizializzazione delle variabili:

- *series*: mantiene il conteggio del numero di sottovettori aritmetici massimali trovati.

- *lastDiff*: memorizza la differenza tra i valori consecutivi nel sottovettore corrente. Inizialmente è impostato a $+\infty$ per distinguere il primo conteggio.
- *count*: tiene traccia del numero di differenze consecutive uguali nel sottovettore corrente.

2. **Ciclo principale:** Si itera attraverso l'array A a partire dalla seconda posizione ($i = 2$).

- Per ogni coppia consecutiva di elementi $A[i-1]$ e $A[i]$, si calcola la differenza $currentDiff = A[i] - A[i-1]$
- **Caso 1: La differenza cambia** ($currentDiff \neq last$):
 - Si aggiorna $last = currentDiff$.
 - Si reimposta $count = 1$, poiché il nuovo sottovettore inizia con una sola differenza calcolata.
- **Caso 2: La differenza è uguale a quella precedente** ($currentDiff = last$):
 - Si incrementa $count$ di 1.
 - Se $count = 2$, significa che abbiamo identificato un sottovettore aritmetico (almeno tre elementi consecutivi con la stessa differenza). Si incrementa *series* di 1; eventuali valori successivi che aumentano la lunghezza non verranno considerati.

3. **Risultato:** Alla fine del ciclo, la variabile *series* contiene il numero totale di sottovettori aritmetici massimali trovati, che viene restituito come output.

L'algoritmo scorre l'array A una sola volta, quindi la complessità temporale è $O(n)$.

Esercizio B2 – Radice-foglia – Punti ≥ 10

L'algoritmo `allpath()` utilizza un approccio ricorsivo per esplorare tutti i cammini radice-foglia di un albero binario. La funzione principale inizializza una struttura *stack* per memorizzare il cammino corrente e avvia la funzione ricorsiva ausiliaria `apRec`, passando la radice dell'albero come parametro iniziale.

La funzione `apRec()` si occupa di visitare ogni nodo dell'albero. Quando viene chiamata su un nodo t , aggiunge il nodo corrente alla pila *stack*, che rappresenta il cammino parziale dalla radice al nodo corrente. Se il nodo t è una foglia, ovvero non ha figli né a sinistra né a destra, il contenuto della pila *stack*, che rappresenta ora il cammino completo dalla radice alla foglia, viene stampato. In seguito, la funzione chiama ricorsivamente se stessa sui figli sinistro e destro del nodo corrente, continuando l'esplorazione dell'albero.

Dopo aver visitato i sottoalberi del nodo corrente, la funzione esegue un'operazione di backtracking rimuovendo il nodo corrente dalla pila *stack*. Questo garantisce che la struttura della pila rifletta correttamente il cammino attivo quando si ritorna al nodo padre. Questo meccanismo di backtracking è fondamentale per assicurare che ogni cammino radice-foglia venga stampato esattamente una volta.

```
allpath(TREE T)
```

```
    STACK stack = Stack()
    apRec(T, stack)
```

```
apRec(TREE t, STACK stack)
```

```
    if t ≠ nil then
        stack.push(t)
        if t.left() == nil and t.right() == nil then
            | print stack                                % Raggiunta una foglia, stampo il cammino
        apRec(t.left(), stack)
        apRec(t.right(), stack)
        stack.pop()
```

L'algoritmo è corretto poiché visita ogni nodo dell'albero esattamente una volta e, per ogni foglia raggiunta, stampa il cammino corrispondente dalla radice a quella foglia. Il backtracking permette di gestire correttamente il cammino durante la visita ricorsiva, evitando conflitti tra percorsi diversi.

Dal punto di vista computazionale, l'algoritmo visita ogni nodo una sola volta, ma la stampa dei cammini contribuisce in modo significativo alla complessità complessiva. È utile considerare diversi casi per comprendere meglio l'impatto della struttura dell'albero sulla complessità:

- **Albero degenerato:** Se l'albero è degenerato, ossia una lista collegata, esiste un unico cammino radice-foglia, poiché c'è una sola foglia. In questo caso, l'algoritmo ha un costo totale di $O(n)$, dove n è il numero di nodi dell'albero.

- **Albero bilanciato:** In un albero completo bilanciato, esistono approssimativamente $n/2$ foglie, ciascuna delle quali corrisponde a un cammino dalla radice alla foglia. Poiché la profondità di un albero bilanciato è $O(\log n)$, ogni cammino ha lunghezza $O(\log n)$. Il costo complessivo dell'algoritmo in questo caso è quindi $O(n \log n)$, dato che la stampa di $n/2$ cammini di lunghezza $O(\log n)$ domina il costo del semplice attraversamento dell'albero.
- **Caso generale:** È possibile costruire un albero in cui la stampa raggiunge una complessità totale di $O(n^2)$. Consideriamo un albero composto da $n/2$ nodi disposti a lista collegata, con in fondo a questa lista un sottoalbero completo con $n/2$ nodi. In questa configurazione, ci sono circa $n/4$ foglie, e ciascun cammino ha una lunghezza di $n/2 + O(\log n)$, poiché include il percorso nella lista e la profondità del sottoalbero completo. La stampa di $n/4$ cammini di lunghezza $n/2 + O(\log n)$ produce una complessità totale di $O(n^2)$.

L'algoritmo non può impiegare più di $O(n^2)$, perché mal che vada deve stampare un cammino di lunghezza $O(n)$ per ognuno degli n nodi. Identificare tuttavia un albero in cui questo accade è interessante, perché mostra che tale complessità può essere raggiunta.

Esercizio B3 – Quante parentesi – Punti ≥ 12

L'algoritmo calcola il numero di parentesizzazioni corrette utilizzando la programmazione dinamica, sfruttando la relazione tra le parentesizzazioni e i numeri di Catalan. Una parentesizzazione corretta può essere costruita seguendo la logica:

- Si inizia con una parentesi aperta.
- La parentesi aperta è seguita da una parentesizzazione valida di lunghezza i (con i compreso tra 0 e $k-1$).
- La parentesi chiusa completa la parentesizzazione, lasciando $k-i-1$ coppie da completare.

Le parentesizzazioni di lunghezza i e quelle di lunghezze $ki-i-1$ si combinano insieme; quindi questi numeri vanno moltiplicati insieme.

La formula è quindi:

$$DP[k] = \begin{cases} 1 & k = 0 \text{ or } k = 1 \\ \sum_{i=0}^{k-1} DP[i] \cdot DP[k-i-1] & k > 1 \end{cases}$$

countPars(int n)

```
int [] DP = new int[0...n]
DP[0] = DP[1] = 1
for k = 2 to n do
    DP[k] = 0
    for i = 0 to k-1 do
        DP[k] = DP[k] + DP[i] * DP[k-i-1]
return DP[n]
```

L'output per n che va da 1 a 10 è quindi [1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16796].

La complessità è $O(n^2)$, poiché per ogni n calcoliamo una somma che coinvolge $n-1$ elementi.